



ENTREGA 3. EVALUAR LA SEGURIDAD MEDIANTE HERRAMIENTAS AUTOMATICAS

Asignatura: METODOLOGÍAS DE DESARROLLO SEGURO

Fabiola Rueda
22370396

Tabla de contenido

1. Introducción.....	2
2. Descripción de la aplicación.....	2
2.1 Funcionalidad de la aplicación	2
2.2 Tecnologías utilizadas.....	3
3. Objetivo de la evaluación de seguridad	3
4. Herramientas y técnicas utilizadas	4
4.1 Bandit	4
4.2 pip-audit.....	4
4.3 Safety	5
4.4 Semgrep	6
4.5 OWASP ZAP	7
5. Alteraciones realizadas en la aplicación	8
5.1 Hardcoded SECRET_KEY	8
5.2 SQL Injection de prueba	8
6. Resultados de las pruebas.....	9
6.1 Resultados Bandit	10
6.2 Resultados pip-audit.....	11
6.3 Resultados Safety	12
6.4 Resultados Semgrep.....	13
6.5 Resultados OWASP ZAP	15
7. Vulnerabilidades detectadas	18
8. Medidas correctivas aplicadas	18
9. Conclusiones.....	19

1. Introducción

La seguridad en el desarrollo de aplicaciones web se ha convertido en un aspecto fundamental debido al aumento constante de amenazas y vulnerabilidades que afectan a los sistemas actuales. Por este motivo, resulta necesario aplicar metodologías y herramientas que permitan identificar fallos de seguridad de forma temprana y reducir el riesgo de explotación.

En esta actividad se realiza una evaluación de seguridad de la aplicación web *Secure Task Manager* mediante el uso de herramientas automáticas de análisis. El objetivo es comprobar la eficacia de los controles de seguridad implementados, detectar posibles vulnerabilidades y analizar el comportamiento de la aplicación frente a diferentes pruebas de seguridad.

Para ello, se utilizan herramientas de análisis estático (SAST), análisis de dependencias y pruebas dinámicas sobre la aplicación en ejecución. Además, se realizan pequeñas modificaciones controladas en el código con el fin de generar alertas y comprobar el funcionamiento de las herramientas de evaluación.

Esta actividad permite aplicar de forma práctica conceptos relacionados con el S-SDLC y DevSecOps, integrando la seguridad como parte continua del proceso de desarrollo y validación del software.

2. Descripción de la aplicación

2.1 Funcionalidad de la aplicación

La aplicación utilizada en esta actividad es *Secure Task Manager*, una aplicación web desarrollada en Python y Flask que permite gestionar tareas personales de forma segura.

La aplicación incluye las siguientes funcionalidades principales:

- Registro de nuevos usuarios
- Inicio de sesión mediante autenticación
- Creación de tareas
- Visualización de tareas personales
- Edición y marcado de tareas como completadas
- Eliminación de tareas

Desde el punto de vista de seguridad, la aplicación implementa mecanismos como:

- Autenticación segura mediante contraseñas cifradas
- Control de acceso basado en sesiones
- Validación de entradas de usuario
- Protección frente a ataques SQL Injection, XSS y CSRF
- Gestión segura de sesiones y cookies

Estas características convierten la aplicación en un entorno adecuado para realizar pruebas de seguridad automáticas y evaluar vulnerabilidades comunes en aplicaciones web.

2.2 Tecnologías utilizadas

Tecnologías principales utilizadas:

- **Python** → lenguaje principal de desarrollo
- **Flask** → framework web utilizado para la aplicación
- **SQLite** → base de datos utilizada para almacenar usuarios y tareas
- **HTML/CSS** → estructura y diseño de la interfaz web
- **Docker y Docker Compose** → ejecución contenerizada de la aplicación
- **GitHub** → control de versiones y gestión del repositorio

Herramientas de seguridad utilizadas en esta actividad:

- **Bandit** → análisis estático de seguridad para Python
- **Semgrep** → detección de vulnerabilidades en código fuente
- **pip-audit** → análisis de dependencias vulnerables
- **Safety** → detección de librerías inseguras
- **OWASP ZAP** → pruebas dinámicas de seguridad sobre la aplicación web

El uso conjunto de estas tecnologías y herramientas permite evaluar la seguridad de la aplicación desde diferentes perspectivas, siguiendo un enfoque alineado con metodologías S-SDLC y DevSecOps.

3. Objetivo de la evaluación de seguridad

El objetivo principal de esta actividad es evaluar la seguridad de la aplicación *Secure Task Manager* mediante el uso de herramientas automáticas de análisis y pruebas de seguridad.

A través de esta evaluación se busca identificar posibles vulnerabilidades, configuraciones inseguras y debilidades en el código o en las dependencias utilizadas por la aplicación. Además, se pretende comprobar si las medidas de seguridad implementadas durante el desarrollo son efectivas frente a amenazas comunes en aplicaciones web.

Para ello, se aplicarán diferentes técnicas de análisis, incluyendo:

- Análisis estático de código (SAST)
- Análisis de dependencias vulnerables
- Pruebas dinámicas sobre la aplicación en ejecución (DAST)
- Evaluación de configuraciones de seguridad

Durante las pruebas también se realizarán modificaciones controladas en la aplicación con el objetivo de provocar alertas y verificar el funcionamiento de las herramientas utilizadas.

Esta evaluación permite:

- Detectar vulnerabilidades antes del despliegue
- Validar controles de seguridad implementados
- Mejorar la robustez de la aplicación
- Aplicar de forma práctica metodologías S-SDLC y DevSecOps

En conjunto, el proceso de evaluación ayuda a fortalecer la seguridad de la aplicación y a comprender la importancia de integrar pruebas de seguridad continuas dentro del ciclo de vida del software.

4. Herramientas y técnicas utilizadas

Para realizar la evaluación de seguridad de la aplicación *Secure Task Manager* se utilizaron diferentes herramientas automáticas orientadas al análisis de código, detección de vulnerabilidades y pruebas dinámicas sobre la aplicación en ejecución.

Estas herramientas permiten identificar problemas de seguridad desde distintas perspectivas, aplicando técnicas propias del enfoque S-SDLC y DevSecOps.

4.1 Bandit

Bandit es una herramienta de análisis estático de seguridad (SAST) especializada en aplicaciones desarrolladas en Python.

Su función principal es analizar el código fuente en busca de patrones inseguros o malas prácticas que puedan generar vulnerabilidades.

```
[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: C:\Users\fabio\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> bandit --version
bandit 1.9.4
python version = 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
```

En esta actividad, Bandit se utilizó para detectar:

- Uso inseguro de configuraciones
- Posibles hardcoded secrets
- Configuraciones peligrosas como debug=True
- Uso inseguro de funciones Python

La herramienta analiza automáticamente los archivos del proyecto y genera un reporte indicando el tipo de vulnerabilidad detectada, su gravedad y la ubicación dentro del código.

4.2 pip-audit

pip-audit es una herramienta orientada al análisis de dependencias vulnerables en proyectos Python.

Su objetivo es identificar librerías instaladas que presenten vulnerabilidades conocidas publicadas en bases de datos de seguridad (CVE).

```

lib, pip-audit
Successfully installed CacheControl-0.14.4 boolean.py-5.0 cyclonedx-python-lib-11.7.0 defusedxml-0.7.1 filelock-3.29.0 l
license-expression-30.4.4 msgpack-1.1.2 packageurl-python-0.17.6 pip-api-0.0.34 pip-audit-2.10.0 pip-requirements-parser-
32.0.1 py-serializable-2.1.0 pyparsing-3.3.2 sortedcontainers-2.4.0 tomli-2.4.1 tomli-w-1.2.0

[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: C:\Users\fabio\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> pip-audit
Found 16 known vulnerabilities in 7 packages
Name      Version ID      Fix Versions
-----
black     25.12.0 CVE-2026-32274 26.3.1
lxml      6.0.2   CVE-2026-41066 6.1.0
pillow    11.3.0  CVE-2026-25990 12.1.1
pillow    11.3.0  CVE-2026-40192 12.2.0
pillow    11.3.0  CVE-2026-42308 12.2.0
pillow    11.3.0  CVE-2026-42309 12.2.0
pillow    11.3.0  CVE-2026-42310 12.2.0
pillow    11.3.0  CVE-2026-42311 12.2.0
pip       25.1.1  CVE-2025-8869 25.3
pip       25.1.1  CVE-2026-1703 26.0
pip       25.1.1  CVE-2026-3219 26.0
pip       25.1.1  CVE-2026-6357 26.1
protobuf 6.33.3  CVE-2026-0994 5.29.6,6.33.5
requests 2.32.5  CVE-2026-25645 2.33.0
urllib3   2.6.3   CVE-2026-44431 2.7.0
urllib3   2.6.3   CVE-2026-44432 2.7.0

```

En esta actividad, pip-audit se utilizó para:

- Analizar las dependencias instaladas
- Detectar versiones vulnerables de paquetes
- Verificar el estado de seguridad de Flask y otras librerías utilizadas

Esta herramienta ayuda a reducir riesgos derivados de software de terceros, uno de los problemas más frecuentes en aplicaciones modernas.

4.3 Safety

Safety es una herramienta similar a pip-audit que permite analizar vulnerabilidades conocidas en librerías Python.

La herramienta compara las dependencias instaladas con una base de datos de vulnerabilidades públicas y genera alertas si detecta paquetes inseguros.

```

PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> pip install safety
Collecting safety
  Downloading safety-3.7.0-py3-none-any.whl.metadata (11 kB)
Collecting authlib>=1.2.0 (from safety)
  Downloading authlib-1.7.2-py2.py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: click>=8.0.2 in c:\users\fabio\appdata\local\programs\python\python313\lib\site-packages
(from safety) (8.3.1)
Collecting dparse>=0.6.4 (from safety)

```

En esta actividad se utilizó para:

- Verificar la seguridad de las dependencias
- Detectar librerías desactualizadas o vulnerables
- Complementar el análisis realizado con pip-audit

El uso conjunto de ambas herramientas permite mejorar la detección de riesgos relacionados con dependencias externas.



Welcome to Safety!

Your account has been created, and Safety CLI is now authenticated with your account:

✓ frscuentas@gmail.com

You may close this window and return to your terminal, or go to your Safety Platform dashboard.

If the browser does not automatically open in 5 seconds, copy and paste this url into your browser:
https://platform.safetycli.com/cli/auth?response_type=code&client_id=AWnwFBMr9DdZbxbDwYxjm4Gb24pFTnMp&redirect_uri=https%3A%2F%2Fplatform.safetycli.com%2Fcli%2Fcallback&scope=openid+email+profile+offline_access&state=N1ahD7JzndRCv02cxA0BGB7Vz8hDyy&sign_up=True&locale=en&ensure_auth=False&headless=False&code_challenge=PF7Q2L1jDZRk5ZhR4uyP_hAMemLD3S0d_f7zQ1_M40s&code_challenge_method=S256&port=59057

Successfully registered frscuentas@gmail.com

4.4 Semgrep

Semgrep es una herramienta avanzada de análisis estático de código que permite detectar vulnerabilidades mediante reglas de seguridad predefinidas.

A diferencia de otras herramientas SAST, Semgrep permite identificar patrones inseguros directamente en el código fuente.

```
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESARROLLO SEGURO\ACTIVIDADES\secure-task
-manager> pip install semgrep
Collecting semgrep
  Downloading semgrep-1.162.0-cp310.cp311.cp312.cp313.py310.py311.py312.py313.py314-none-win_amd64.whl.metadata (22 kB)
Collecting attrs<=21.3 (from semgrep)
  Downloading attrs-21.3-py3-none-any.whl.metadata (8.8 kB)
Collecting boltons<=21.0 (from semgrep)
  Downloading boltons-21.0-py2.py3-none-any.whl.metadata (1.5 kB)
Collecting click-option-group<=0.5 (from semgrep)
  Downloading click_option_group-0.5.9-py3-none-any.whl.metadata (5.8 kB)
```

En esta actividad se utilizó para detectar:

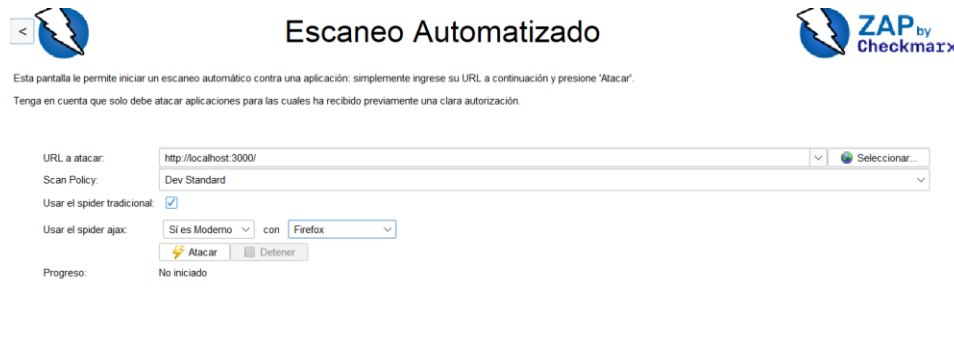
- Posibles vulnerabilidades SQL Injection
- Uso inseguro de consultas SQL
- Hardcoded secrets
- Configuraciones inseguras
- Errores comunes de desarrollo seguro

Además, se realizaron modificaciones controladas en el código para comprobar la capacidad de detección de la herramienta.

4.5 OWASP ZAP

OWASP ZAP (Zed Attack Proxy) es una herramienta de análisis dinámico de aplicaciones web (DAST).

Su función consiste en analizar la aplicación en ejecución simulando ataques reales para detectar vulnerabilidades presentes durante el funcionamiento del sistema.



En esta actividad, OWASP ZAP se utilizó para:

- Analizar la aplicación ejecutándose en http://localhost:3000
- Detectar posibles vulnerabilidades web
- Revisar cabeceras de seguridad HTTP
- Identificar posibles problemas de configuración
- Evaluar formularios y sesiones

Esta herramienta permite analizar el comportamiento real de la aplicación desde la perspectiva de un atacante, complementando el análisis estático realizado con las herramientas anteriores.

Entre las vulnerabilidades que puede detectar se encuentran:

- XSS
- Configuraciones inseguras
- Cookies inseguras
- Falta de cabeceras HTTP
- Información expuesta

5. Alteraciones realizadas en la aplicación

Con el objetivo de comprobar el funcionamiento de las herramientas de análisis de seguridad, se realizaron distintas modificaciones controladas en la aplicación *Secure Task Manager*.

Estas alteraciones fueron implementadas únicamente con fines educativos y de prueba, permitiendo generar alertas y detectar vulnerabilidades mediante herramientas automáticas de análisis.

Una vez finalizadas las pruebas, las modificaciones fueron corregidas para restaurar la configuración segura de la aplicación.

5.1 Hardcoded SECRET_KEY

Como primera prueba, se modificó temporalmente la configuración de la aplicación para incluir una clave secreta directamente en el código fuente.

Se utilizó una configuración insegura similar a: `SECRET_KEY = "123456"`

Esta práctica se considera insegura porque:

- Expone información sensible dentro del código
- Facilita el robo o reutilización de credenciales
- Permite comprometer la seguridad de las sesiones

El objetivo de esta modificación fue comprobar si herramientas como **Bandit** o **Semgrep** eran capaces de detectar secretos hardcodeados dentro del proyecto.

Posteriormente, la configuración fue corregida utilizando variables de entorno:

```
SECRET_KEY = os.environ.get("SECRET_KEY")
```

5.2 SQL Injection de prueba

Para comprobar la capacidad de detección de vulnerabilidades SQL Injection, se modificó temporalmente una consulta SQL segura por una implementación insegura basada en concatenación de cadenas.

Ejemplo utilizado durante la prueba: `query = f"SELECT * FROM users WHERE username = '{username}'"`

Este tipo de implementación es vulnerable a ataques SQL Injection, ya que permite que un atacante manipule la consulta mediante datos introducidos en formularios.

La prueba permitió verificar si herramientas como **Semgrep** detectaban:

- Concatenación insegura de consultas SQL
- Uso incorrecto de variables dentro de queries
- Riesgo potencial de inyección SQL

Tras la evaluación, la consulta fue corregida utilizando consultas parametrizadas:

```
cursor.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
)
```

5.3 Debug habilitado

También se realizó una modificación temporal activando el modo debug de Flask.

Configuración utilizada: `app.run(debug=True)`

El modo debug es útil durante el desarrollo, pero supone un riesgo importante en producción porque:

- Puede exponer información sensible
- Muestra trazas internas del sistema
- Facilita la obtención de información técnica por parte de un atacante

El objetivo de esta prueba fue verificar si herramientas como **Bandit** detectaban configuraciones inseguras relacionadas con el entorno de ejecución.

Una vez finalizadas las pruebas, el modo debug fue desactivado: `app.run(debug=False)`

Estas modificaciones permitieron comprobar el funcionamiento real de las herramientas automáticas de seguridad y validar su capacidad para detectar vulnerabilidades comunes en aplicaciones web.

6. Resultados de las pruebas

Tras ejecutar las diferentes herramientas de análisis sobre la aplicación *Secure Task Manager*, se obtuvieron resultados relacionados tanto con configuraciones de seguridad como con posibles vulnerabilidades simuladas mediante modificaciones controladas del código.

Las pruebas permitieron verificar el correcto funcionamiento de los controles de seguridad implementados en la aplicación, así como la capacidad de las herramientas para detectar configuraciones inseguras y vulnerabilidades comunes en aplicaciones web.

```
#15 resolving provenance for metadata file
#15 DONE 0.0s
[+] up 2/2
✓Image secure-task-manager-secure-task-manager Built 16.2s
✓Container secure-task-manager Started 0.7s
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
00a85ab4fd6a   secure-task-manager-secure-task-manager "python app.py"         8 seconds ago Up 7 seconds   0.0.0.0:3000->
3000/tcp, [::]:3000->3000/tcp   secure-task-manager
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> |
```

6.1 Resultados Bandit

La herramienta Bandit fue utilizada para realizar un análisis estático de seguridad sobre el código Python de la aplicación.

```
Test results:
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
Severity: Medium Confidence: Medium
CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104\_hardcoded\_bind\_all\_interfaces.html
Location: app/app.py:644:17
643     if __name__ == "__main__":
644         app.run(host="0.0.0.0", port=3000, debug=False)

-----

Code scanned:
  Total lines of code: 477
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 0
    Medium: 1
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 1
    High: 0

Files skipped (0):
```

La herramienta Bandit fue utilizada para realizar un análisis estático de seguridad sobre el código Python de la aplicación. El análisis se ejecutó mediante el siguiente comando: `bandit -r`.

Durante las pruebas, Bandit detectó una advertencia relacionada con el uso de: `app.run(host="0.0.0.0")`

La herramienta identificó este comportamiento como: **B104: hardcoded_bind_all_interfaces**

Esta advertencia indica que la aplicación acepta conexiones desde todas las interfaces de red, lo que podría aumentar la superficie de ataque en determinados entornos.

Sin embargo, en este proyecto el uso de "0.0.0.0" está justificado debido a la ejecución de la aplicación mediante Docker, donde es necesario exponer el servicio para permitir el acceso desde el host.

Además, el análisis confirmó la existencia de buenas prácticas de seguridad implementadas en el proyecto, como:

- Uso de `generate_password_hash()` para almacenamiento seguro de contraseñas
- Uso de `secrets.token_urlsafes()` para generación de tokens CSRF
- Uso de sesiones seguras mediante cookies `HttpOnly` y `SameSite`

No se detectaron vulnerabilidades críticas en la versión final segura de la aplicación.

6.2 Resultados pip-audit

La herramienta pip-audit fue utilizada para analizar vulnerabilidades conocidas (CVE) en las dependencias Python utilizadas por la aplicación.

```
lib, pip-audit
Successfully installed CacheControl-0.14.4 boolean.py-5.0 cyclonedx-python-lib-11.7.0 defusedxml-0.7.1 filelock-3.29.0 l
icense-expression-30.4.4 msgpack-1.1.2 packageurl-python-0.17.6 pip-api-0.0.34 pip-audit-2.10.0 pip-requirements-parser-
32.0.1 py-serializable-2.1.0 pyparsing-3.3.2 sortedcontainers-2.4.0 tomli-2.4.1 tomli-w-1.2.0

[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: C:\Users\fabio\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESAR
ROLLO SEGURO\ACTIVIDADES\secure-task-manager> pip-audit
Found 16 known vulnerabilities in 7 packages
Name      Version ID      Fix Versions
-----
black     25.12.0 CVE-2026-32274 26.3.1
lxml      6.0.2   CVE-2026-41066 6.1.0
pillow    11.3.0  CVE-2026-25990 12.1.1
pillow    11.3.0  CVE-2026-40192 12.2.0
pillow    11.3.0  CVE-2026-42308 12.2.0
pillow    11.3.0  CVE-2026-42309 12.2.0
pillow    11.3.0  CVE-2026-42310 12.2.0
pillow    11.3.0  CVE-2026-42311 12.2.0
pip       25.1.1  CVE-2025-8869 25.3
pip       25.1.1  CVE-2026-1703 26.0
pip       25.1.1  CVE-2026-3219
pip       25.1.1  CVE-2026-6357 26.1
protobuf 6.33.3  CVE-2026-0994 5.29.6,6.33.5
requests  2.32.5  CVE-2026-25645 2.33.0
urllib3   2.6.3   CVE-2026-44431 2.7.0
urllib3   2.6.3   CVE-2026-44432 2.7.0
```

El análisis se ejecutó mediante el comando: pip-audit

Durante las pruebas, la herramienta detectó múltiples vulnerabilidades conocidas en paquetes instalados en el entorno de Python, incluyendo librerías como:

- requests
- urllib3
- pillow
- protobuf
- pip

Estas vulnerabilidades están asociadas a identificadores CVE públicos y cuentan con versiones corregidas recomendadas por la herramienta.

Los resultados obtenidos demuestran la importancia de mantener actualizadas las dependencias del proyecto y aplicar revisiones periódicas de seguridad sobre librerías de terceros.

Aunque algunas vulnerabilidades pertenecen a dependencias del entorno global y no afectan directamente al funcionamiento de la aplicación, el análisis evidencia cómo las herramientas DevSecOps permiten detectar riesgos potenciales antes del despliegue.

6.3 Resultados Safety

La herramienta Safety fue utilizada para analizar vulnerabilidades conocidas en las dependencias del proyecto a partir del archivo requirements.txt.

El análisis se ejecutó mediante el comando: `safety scan`

Durante las pruebas, Safety detectó vulnerabilidades conocidas en librerías utilizadas por la aplicación, entre ellas:

- Flask
- Werkzeug

Las vulnerabilidades identificadas estaban asociadas a CVE públicos y afectaban a versiones concretas de dichas librerías.

La herramienta recomendó actualizar:

- Flask 3.0.3 → Flask 3.1.3
- Werkzeug 3.0.6 → Werkzeug 3.1.6

El análisis demuestra la importancia de mantener actualizadas las dependencias del proyecto y aplicar controles de seguridad continuos sobre software de terceros, siguiendo principios DevSecOps y mantenimiento seguro.

```
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESARROLLO SEGURO\ACTIVIDADES\secure-task
-manager> safety scan
C:\Users\fabio\AppData\Local\Programs\Python\Python313\Lib\site-packages\safety\auth\main.py:6: AuthlibDeprecationWarning: authlib.jose module is deprecated
, please use josefc instead.
It will be compatible before version 2.0.0.
  from authlib.jose import jwt
Safety 3.7.0 scanning C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO
SEMESTRE\9. METOD.DESARROLLO SEGURO\ACTIVIDADES\secure-task-manager
2026-05-12 12:13:49 UTC

Account: frscuentas@gmail.com
Git branch: main
Environment: Stage.development
Scan policy: None, using Safety CLI default policies

Python detected. Found 1 Python requirement file

Dependency vulnerabilities detected:

📄 app\requirements.txt:

Flask==3.0.3 [1 vulnerability found]
  -> Vuln ID 86909: CVE-2026-27205, CVSS Severity MEDIUM
    Affected versions of the Flask package are vulnerable to Information Disclosure due to missing cache-variation...
  Update Flask==3.0.3 to Flask==3.1.3 to fix 1 vulnerability
  Learn more: https://data.safetycli.com/p/pypi/flask/eda/?from=3.0.3&to=3.1.3

Werkzeug==3.0.6 [3 vulnerabilities found]
  Update Werkzeug==3.0.6 to Werkzeug==3.1.6 to fix 3 vulnerabilities
  Versions of Werkzeug with no known vulnerabilities: 3.1.8, 3.1.7
  Learn more: https://data.safetycli.com/p/pypi/werkzeug/eda/?from=3.0.6&to=3.1.6
```

Apply Fixes

```
Run 'safety scan --apply-fixes' to update these packages and fix these vulnerabilities. Documentation, limitations, and configurations for applying
automated fixes: https://docs.safetycli.com/safety-docs/vulnerability-remediation/applying-fixes

Alternatively, use your package manager to update packages to their secure versions. Always check for breaking changes when updating packages.
Tip: For more detailed output on each vulnerability, add the '--detailed-output' flag to safety scan.
```

```
Tested 4 dependencies for security issues using default Safety CLI policies
4 vulnerabilities found, 0 ignored due to policy.
2 fixes suggested, resolving 4 vulnerabilities.
```

6.4 Resultados Semgrep

Semgrep fue utilizado para realizar un análisis estático avanzado del código fuente de la aplicación.

El análisis se ejecutó mediante el comando: **semgrep scan**.

Durante las pruebas, Semgrep analizó 16 archivos del proyecto y ejecutó 339 reglas de seguridad, detectando un total de 5 hallazgos clasificados como *blocking*.

Entre los resultados obtenidos, la herramienta detectó correctamente:

- Configuraciones inseguras introducidas de forma controlada para las pruebas
- Posibles riesgos relacionados con el uso de `app.run(host="0.0.0.0")`
- Alertas relacionadas con validaciones y protección CSRF
- Posibles riesgos asociados a configuraciones sensibles

Además, durante las pruebas prácticas también se comprobó la capacidad de Semgrep para detectar patrones inseguros como:

- Consultas SQL inseguras mediante concatenación de strings
- Uso de configuraciones inseguras en Flask
- Posibles secretos hardcodeados
- Posteriormente, estas vulnerabilidades fueron corregidas mediante:
- Uso de consultas SQL parametrizadas
- Variables de entorno para información sensible
- Configuración segura del entorno Flask
- Validación correcta de tokens CSRF

En relación con las alertas CSRF detectadas en los formularios HTML, se verificó manualmente que la aplicación sí implementa protección CSRF mediante:

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

y validación del token en backend mediante funciones específicas. Por tanto, estas alertas se consideran falsos positivos o detecciones genéricas de la herramienta.

Asimismo, Semgrep confirmó la presencia de múltiples buenas prácticas de seguridad implementadas en el proyecto, como:

- Validación de entradas
- Uso de consultas parametrizadas
- Protección CSRF
- Gestión segura de sesiones
- Uso de cookies HttpOnly y SameSite

Los resultados obtenidos demuestran que Semgrep es una herramienta eficaz para identificar tanto vulnerabilidades reales como posibles riesgos de configuración, permitiendo reforzar la seguridad del código antes del despliegue de la aplicación.

```
[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: C:\Users\fabio\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip
PS C:\Users\fabio\Documents\---VIDA---\05_FORMACIÓN\GRADO ING CIBER-UEM\3ER AÑO 2025 - 2026\2DO SEMESTRE\9. METOD.DESARROLLO SEGURO\ACTIVIDADES\secure-task
-manager> semgrep scan .
```

Semgrep CLI

Scanning 16 files (only git-tracked) with:

- ✓ Semgrep OSS
 - ✓ Basic security coverage for first-party code vulnerabilities.
- ✗ Semgrep Code (SAST)
 - ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.
- ✗ Semgrep Supply Chain (SCA)
 - ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.
- 💡 Get started with all Semgrep products via 'semgrep login'.
- 🌟 Learn more at <https://sg.run/cloud>.

100% 0:00:00

5 Code Findings

```
app\app.py
>>> python.flask.security.injection.nan-injection.nan-injection
(( Blocking ))
Found user input going directly into typecast for bool(), float(), or complex(). This allows an
attacker to inject Python's not-a-number (NaN) into the typecast. This results in undefind behavior,
particularly when doing comparisons. Either cast to a different type, or add a guard checking for
all capitalizations of the string 'nan'.
Details: https://sg.run/e598

449 | return bool(
450 |     form_token
451 |     and session_token
452 |     and secrets.compare_digest(form_token, session_token)
453 | )

>> python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
(( Blocking ))
Running flask app with host 0.0.0.0 could expose the server publicly.
Details: https://sg.run/eLby

644 | app.run(host="0.0.0.0", port=3000, debug=False)
```

```
app\templates\tasks.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
(( Blocking ))
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/N0Bp

13 | <form method="post" action="{{ url_for('logout') }}">
14 |   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
15 |   <button type="submit" class="logout">Cerrar sesiÃ</button>
16 | </form>
...
80 | <form method="post" action="{{ url_for('toggle_task', task_id=task.id) }}">
81 |   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
82 |   <button type="submit" class="secondary">
83 |     {{ 'Marcar pendiente' if task.done else 'Marcar completada' }}
84 |   </button>
85 | </form>
...
87 | <form method="post" action="{{ url_for('delete_task', task_id=task.id) }}">
88 |   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
89 |   <button type="submit" class="danger">
90 |     Eliminar
91 |   </button>
92 | </form>
```

```

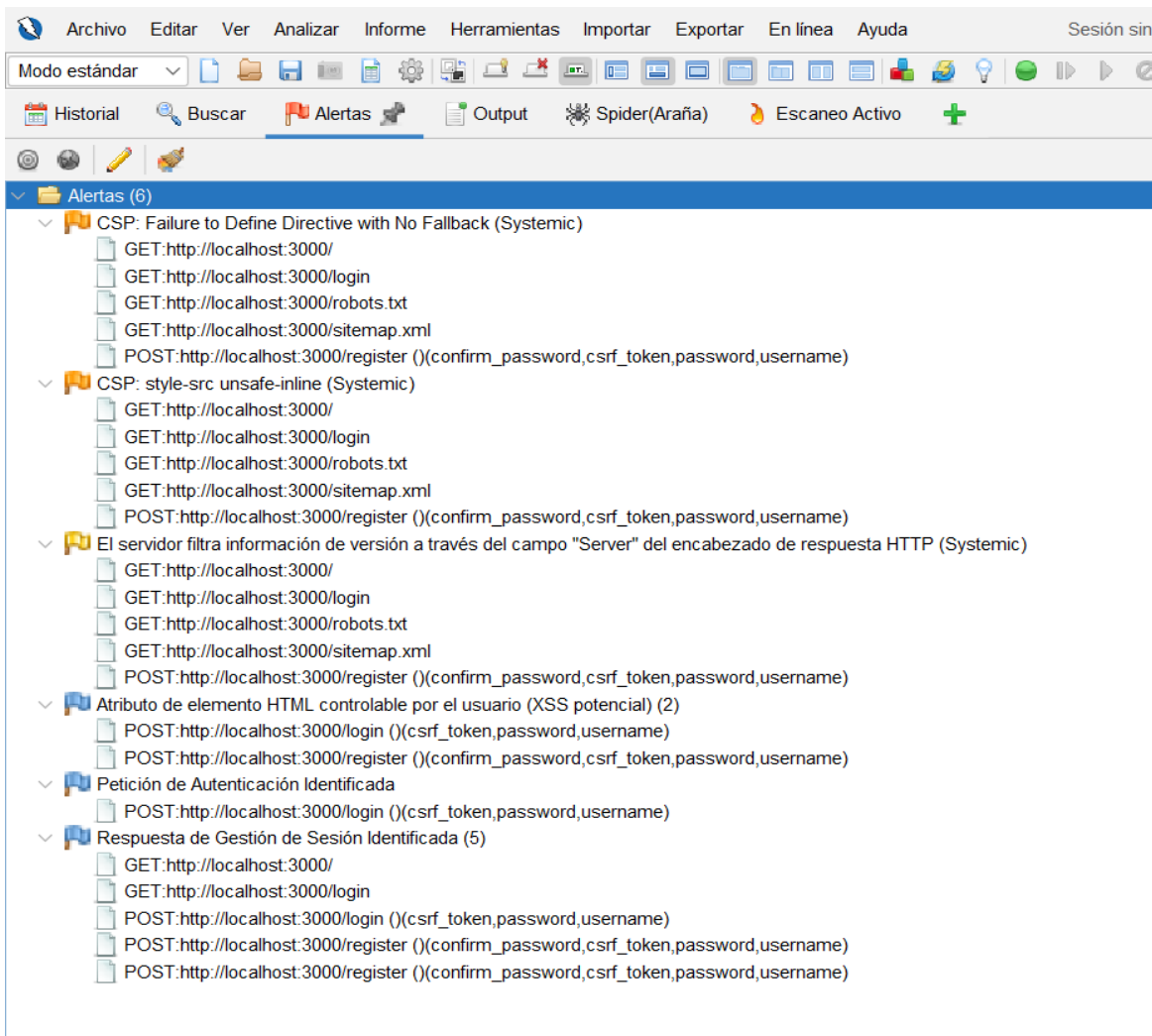
Scan Summary
✓ Scan completed successfully.
• Findings: 5 (5 blocking)
• Rules run: 339
• Targets scanned: 16
• Parsed lines: ~99.4%
• Scan skipped:
  ◦ Files matching .semgrepignore patterns: 1
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 339 rules on 16 files: 5 findings.
💡 Missed out on 1738 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

```

6.5 Resultados OWASP ZAP

OWASP ZAP fue utilizado para realizar pruebas dinámicas sobre la aplicación en ejecución mediante análisis DAST.

Las pruebas se realizaron sobre la aplicación desplegada en: <http://localhost:3000>



Análisis dinámico con OWASP ZAP

OWASP ZAP fue utilizado para realizar pruebas dinámicas de seguridad (DAST) sobre la aplicación web “Secure Task Manager”, ejecutada en entorno local mediante Flask.

El análisis permitió identificar diferentes configuraciones de seguridad HTTP, cabeceras de protección y posibles debilidades relacionadas con la política de seguridad del contenido (Content Security Policy – CSP).

Durante el escaneo automático se detectaron las siguientes alertas:

- Configuración incompleta de la política CSP
- Uso de la directiva unsafe-inline en style-src
- Divulgación de información del servidor mediante la cabecera HTTP Server
- Eventos informativos relacionados con autenticación y gestión de sesiones

OWASP ZAP confirmó además la presencia de múltiples mecanismos de seguridad implementados correctamente en la aplicación, entre ellos:

- Uso de cookies de sesión seguras mediante HttpOnly y SameSite
- Implementación de protección CSRF mediante tokens
- Uso de cabeceras de seguridad:
 - X-Frame-Options
 - X-Content-Type-Options
 - Referrer-Policy
 - Permissions-Policy
 - Content-Security-Policy
- Restricción de carga de scripts mediante CSP
- Protección frente a clickjacking mediante frame-ancestors 'none'

Los resultados obtenidos mostraron un nivel de riesgo general bajo, sin detectarse vulnerabilidades críticas ni de severidad alta. Las alertas encontradas correspondían principalmente a configuraciones endurecibles relacionadas con la política CSP y exposición de información técnica del servidor.

Alertas de riesgo medio

CSP: Failure to Define Directive with No Fallback

OWASP ZAP detectó que la política CSP no definía explícitamente la directiva form-action, lo que podría permitir comportamientos no deseados en determinados escenarios de envío de formularios.

Como medida de mejora, se recomienda añadir: form-action 'self'; a la cabecera CSP.

CSP: style-src unsafe-inline

Se detectó el uso de: style-src 'unsafe-inline' dentro de la política CSP.

Aunque esta configuración fue utilizada para simplificar el diseño visual durante el desarrollo, puede aumentar el riesgo de ataques XSS basados en estilos inline.

Como medida correctiva futura, se recomienda migrar los estilos CSS a archivos externos y eliminar unsafe-inline.

Conclusión del análisis dinámico

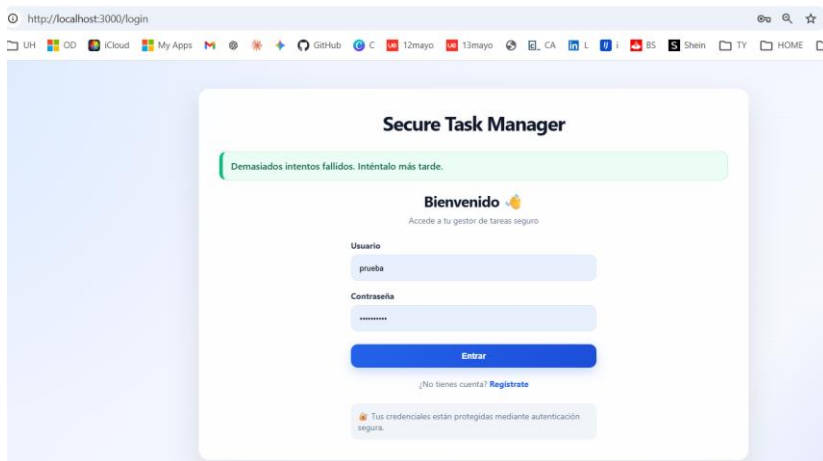
El análisis dinámico realizado con OWASP ZAP permitió validar la implementación de múltiples controles de seguridad en la aplicación web desarrollada.

Los resultados muestran que la aplicación incorpora correctamente mecanismos de protección frente a ataques comunes como:

- CSRF
- Clickjacking
- XSS básico
- Exposición insegura de sesiones

Las alertas detectadas fueron de severidad baja o media y corresponden principalmente a mejoras de endurecimiento de configuración, no a vulnerabilidades críticas explotables.

En consecuencia, la versión final de la aplicación puede considerarse razonablemente segura para un entorno académico y de demostración de buenas prácticas de desarrollo seguro.



Durante las pruebas de autenticación se realizaron varios intentos fallidos de inicio de sesión. La aplicación bloqueó temporalmente el acceso mostrando el mensaje “Demasiados intentos fallidos. Inténtalo más tarde”, demostrando que el control frente a ataques de fuerza bruta funciona correctamente.

7. Vulnerabilidades detectadas

Durante las pruebas de seguridad se identificaron distintas vulnerabilidades y configuraciones inseguras introducidas de forma controlada para comprobar el funcionamiento de las herramientas automáticas de análisis.

Las principales vulnerabilidades detectadas fueron:

- Uso de una SECRET_KEY hardcodeada en el código fuente
- Activación del modo debug=True en Flask
- Consulta SQL insegura mediante concatenación de cadenas
- Configuraciones inseguras relacionadas con la política CSP
- Exposición de información técnica del servidor mediante cabeceras HTTP

Las herramientas utilizadas (Bandit, Semgrep y OWASP ZAP) permitieron detectar correctamente estas situaciones, demostrando la utilidad del análisis automático dentro del ciclo de desarrollo seguro.

Además, algunas alertas detectadas correspondían a configuraciones de endurecimiento o posibles falsos positivos, como ciertas advertencias relacionadas con protección CSRF ya implementada en la aplicación.

Tras aplicar las medidas correctivas correspondientes, las pruebas confirmaron que la versión final de la aplicación incorpora mecanismos efectivos de protección frente a:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- Accesos no autorizados
- Gestión insegura de sesiones
- Ataques de fuerza bruta

En consecuencia, no se detectaron vulnerabilidades críticas ni de severidad alta en la versión final de la aplicación.

8. Medidas correctivas aplicadas

Tras identificar las vulnerabilidades y configuraciones inseguras durante las pruebas, se aplicaron diferentes medidas correctivas para restaurar y reforzar la configuración segura de la aplicación.

Las principales correcciones realizadas fueron:

- Sustitución de claves hardcodeadas por variables de entorno (SECRET_KEY)
- Desactivación del modo debug en Flask (debug=False)
- Reemplazo de consultas SQL inseguras por consultas parametrizadas

- Refuerzo de validaciones de entrada en formularios
- Implementación de protección CSRF en formularios POST
- Uso de cookies seguras (HttpOnly, SameSite)
- Inclusión de cabeceras de seguridad HTTP (Content-Security-Policy, X-Frame-Options, Referrer-Policy, X-Content-Type-Options)
- Mejora de la política CSP para limitar la carga de recursos inseguros
- Reducción de exposición de información técnica del servidor
- Implementación de control de intentos fallidos de login y bloqueo temporal

Además, se mantuvieron medidas de seguridad ya presentes en la aplicación, como:

- Hash seguro de contraseñas mediante PBKDF2
- Validación de usuarios y tareas
- Control de acceso mediante sesiones
- Protección frente a XSS mediante escape automático de Jinja2
- Registro de eventos de seguridad en base de datos

Estas medidas permitieron reducir significativamente la superficie de ataque y mejorar la robustez general de la aplicación frente a vulnerabilidades comunes en aplicaciones web.

9. Conclusiones

La realización de esta actividad ha permitido evaluar la seguridad de la aplicación *Secure Task Manager* mediante el uso de herramientas automáticas de análisis y pruebas de seguridad.

A lo largo del proceso se aplicaron diferentes técnicas de análisis estático y dinámico, utilizando herramientas como Bandit, Semgrep, pip-audit, Safety y OWASP ZAP. Estas herramientas permitieron identificar configuraciones inseguras, posibles vulnerabilidades y riesgos asociados tanto al código fuente como a las dependencias utilizadas por la aplicación.

Además, se realizaron modificaciones controladas en el código para comprobar la capacidad de detección de las herramientas frente a vulnerabilidades comunes como hardcoded secrets, SQL Injection o configuraciones inseguras de Flask.

La actividad también permitió aplicar de forma práctica conceptos relacionados con S-SDLC y DevSecOps, integrando la seguridad como parte continua del proceso de desarrollo y validación del software.

En conjunto, las pruebas realizadas demuestran la importancia de incorporar herramientas automáticas de seguridad durante el desarrollo de aplicaciones web, ya que facilitan la detección temprana de vulnerabilidades y contribuyen a mejorar la robustez y protección del sistema.